MATH2349 Data Wrangling

Practical Assessment 2

Tung Duc Nguyen, Hoang Nguyen

26/10/2026

1. Students' Details

Group Information

Student Name	Student Number	Contribution (%)
Hoang Nguyen	s4280335	50%
Tung Duc Nguyen	s4275232	50%

2. Required Packages

The following R packages are required to reproduce this report.

We used dplyr (Wickham et al. 2026), tidyr (Wickham, Vaughan and Girlich 2025), readr (Wickham, Hester and Bryan 2026), stringr (Wickham 2025), outliers (Komsta 2022), MVN (Korkmaz, Goksuluk and Zararsiz 2014) and forecast (Hyndman and Khandakar 2008).

```
knitr::opts_chunk$set (out.width = "90%")
# Load necessary packages
library(dplyr)
library(tidyr)
library(readr)
library(outliers)
library(MVN)
# Add other packages as required
library(forecast)
library(stringr)
```

3. Executive Summary

This report prepares two World Bank datasets, GDP per capita and life expectancy at birth, to explore how a country's wealth relates to how long its people live.

- **Data and subsetting:** Both datasets cover 266 countries and regions from 1960 to 2025 and as required, we randomly kept 95% of each (252 rows), using a student number as the seed.
- **Tidying and merging:** Each year was stored in a separate column, so we reshaped both datasets to one row per country per year. We joined them on country code and year, then used the World Bank's country information to remove 44 regional groups such as "World", leaving 194 countries.
- **Data types:** We converted Year to whole numbers, Region to a factor and Income Group to an ordered factor (Low to High income). We also created an ordered Life Expectancy Group (Below 60 to 80+).
- **Missing values and errors:** GDP per capita contained approximately 18% missing observations, making the treatment decision challenging. Mean imputation could distort differences across countries and historical periods, while deletion could reduce representativeness. We therefore examined missingness by year and country before selecting complete-case deletion.
- **Outliers:** Boxplots, z-scores and a two-variable check found many extreme values, such as very wealthy Monaco and very low life expectancy during wars and famines, like Cambodia in 1975–1978. We retained these observations because they were not confirmed errors, although some remain uncertain.
- **Transformation:** GDP per capita was strongly right-skewed, so after comparing five transformations, we applied a base-10 log, which made GDP nearly symmetric (skewness 4.48 to 0.10) and turned its curved relationship with life expectancy into a roughly straight line (correlation 0.53 to 0.84).
- **Limitations:** Deleting incomplete rows removed about 17% of observations, mostly from the 1960s and poorer countries, and GDP is not adjusted for inflation.

4. Data

Two datasets were obtained from the World Bank's World Development Indicators database (World Bank 2026a, 2026b). **Dataset 1: GDP per capita (current US$)**
Source: https://data.worldbank.org/indicator/NY.GDP.PCAP.CD (https://data.worldbank.org/indicator/NY.GDP.PCAP.CD) This dataset measures gross domestic product per person in current US dollars. It includes country names, country codes, indicator information, and annual GDP per capita values from 1960 to 2025. **Dataset 2: Life expectancy at birth, total (years)**
Source: https://data.worldbank.org/indicator/SP.DYN.LE00.IN (https://data.worldbank.org/indicator/SP.DYN.LE00.IN) This dataset estimates the number of years a newborn would live if current mortality patterns remained unchanged and it contains the same geographical identifiers and annual reporting structure. Both datasets contain 266 country, economy, and regional aggregate records and share the Country Code variable and reporting years, allowing them to be merged after reshaping into long format. The datasets are initially presented in wide format, with individual years stored as separate columns. This structure will be addressed during the tidying stage.

```
# Import the World Bank datasets
gdp_raw <- read_csv("API_NY.GDP.PCAP.CD_DS2_en_csv_v2_1529.csv", skip = 4, show_col_types = FALSE)
```

```
## New names:
## • `` -> `...71`
```

```
life_raw <- read_csv("API_SP.DYN.LE00.IN_DS2_en_csv_v2_1528.csv", skip = 4, show_col_types = FALSE)
```

```
## New names:
## • `` -> `...71`
```

```
# Check the dimensions of both datasets
dim(gdp_raw)
```

```
## [1] 266  71
```

```
dim(life_raw)
```

```
## [1] 266  71
```

```
# Preview observations from each dataset
head(gdp_raw[, 1:4], 2)
```

Country Name <chr>	Country Code <chr>	Indicator Name <chr>	Indicator Code <chr>
Aruba	ABW	GDP per capita (current US$)	NY.GDP.PCAP.CD
Africa Eastern and Southern	AFE	GDP per capita (current US$)	NY.GDP.PCAP.CD

2 rows

```
head(life_raw[, 1:4], 2)
```

Country Name <chr>	Country Code <chr>	Indicator Name <chr>	Indicator <chr>
Aruba	ABW	Life expectancy at birth, total (years)	SP.DYN.L
Africa Eastern and Southern	AFE	Life expectancy at birth, total (years)	SP.DYN.L

2 rows

```
# Show the first six and last two column names
head(names(gdp_raw))
```

```
## [1] "Country Name"   "Country Code"   "Indicator Name" "Indicator Code"
## [5] "1960"           "1961"
```

```
tail(names(gdp_raw), 2)
```

```
## [1] "2025"  "...71"
```

We used `read_csv()` from the readr package to import both World Bank datasets into R. The `skip = 4` argument was applied to exclude the preliminary metadata lines. We then used `dim()` to examine the dimensions of each dataset and `head()` to preview the first six observations. The results showed that both datasets contain 266 observations and 71 variables. The preview confirmed that the datasets were imported successfully and contain matching geographical identifiers, including Country Name and Country Code. These identifiers will support the subsequent merging process. Both datasets contain the same columns:

- **Country Name** (character): name of the country, economy or regional group.
- **Country Code** (character): three-letter World Bank code. This is the common key used to join the two datasets.
- **Indicator Name** (character): name of the measure.
- **Indicator Code** (character): World Bank code for the measure.
- **1960 to 2025** (numeric): one for each year, holding GDP per capita in US dollars or life expectancy in years and Cells are blank where no value was reported.

```
# Randomly sample 95% of rows from each original dataset
set.seed(4280335)
gdp_sample <- gdp_raw[sample(nrow(gdp_raw), size = floor(0.95 * nrow(gdp_raw))),]
life_sample <- life_raw[sample(nrow(life_raw), size = floor(0.95 * nrow(life_raw))),]
# Verify sampled dimensions
dim(gdp_sample)
```

```
## [1] 252  71
```

```
dim(life_sample)
```

```
## [1] 252  71
```

We used `set.seed(4280335)` to ensure that the random sampling process is reproducible. We then applied `sample()` to independently select approximately 95% of the observations from each original dataset, using `floor()` to obtain a whole-number sample size. The results showed that both datasets were reduced from 266 to 252 observations, while retaining all 71 variables. This corresponds to approximately 94.74% of the original observations, satisfying the requirement to randomly sample at least 80% but less than 100% of each dataset. All of the following preprocessing steps will use these sampled datasets.

```
# Convert GDP dataset from wide to long format
gdp_tidy <- gdp_sample %>%
  select(Country_Name = `Country Name`, Country_Code = `Country Code`,
         matches("^[0-9]{4}$")) %>%
  pivot_longer(
    cols = matches("^[0-9]{4}$"),
    names_to = "Year",
    values_to = "GDP_per_capita"
  ) %>%
  mutate(Year = as.integer(Year))
# Convert life expectancy dataset from wide to long format
life_tidy <- life_sample %>%
  select(Country_Name = `Country Name`, Country_Code = `Country Code`,
         matches("^[0-9]{4}$")) %>%
  pivot_longer(
    cols = matches("^[0-9]{4}$"),
    names_to = "Year",
    values_to = "Life_expectancy"
  ) %>%
  mutate(Year = as.integer(Year))
# Check the dimensions after tidying
dim(gdp_tidy)
```

```
## [1] 16632     4
```

```
dim(life_tidy)
```

```
## [1] 16632     4
```

We used `pivot_longer()` from the tidyr package to reshape both datasets from wide to long format, as the original datasets stored individual years as separate columns. Additionally, the year columns were then converted into a single Year variable, while the corresponding values were stored in GDP_per_capita and Life_expectancy. We then used `as.integer()` to convert Year from character to integer and renamed the country identifier columns inside select() to remove the spaces from their names. Moreover, all available years from 1960 to 2025 were retained to preserve the historical observations and allow missing values to

be investigated later during the scanning stage. The results showed that both datasets were reshaped into 16,632 observations and four variables. Each row now represents one geographical record in a specific year, making the datasets suitable for merging using Country_Code and Year.

```
# Check for duplicate country-year keys
gdp_tidy %>% count(Country_Code, Year) %>% filter(n > 1) %>% nrow()
```

```
## [1] 0
```

```
life_tidy %>% count(Country_Code, Year) %>% filter(n > 1) %>% nrow()
```

```
## [1] 0
```

```
# Merge both datasets using Country Code and Year
merged_data <- inner_join(
  gdp_tidy,
  life_tidy %>%
    select(Country_Code, Year, Life_expectancy),
  by = c("Country_Code", "Year")
)
# Check the merged dataset
dim(merged_data)
```

```
## [1] 15708     5
```

```
# Preview the first three merged observations
head(merged_data, 3)
```

Country_Name <chr>	Country_Code <chr>	Year <int>	GDP_per_capita <dbl>	Life_expectancy <dbl>
Myanmar	MMR	1960	25.08478	44.086
Myanmar	MMR	1961	27.26545	44.603
Myanmar	MMR	1962	27.94700	45.264

3 rows

We used count() to count how often each combination of Country_Code and Year appears, and filter(n > 1) to keep any combination appearing more than once. The results returned zero duplicates for both datasets, confirming that each country-year combination is unique. We then applied `inner_join()` from the dplyr package to merge the two datasets using Country_Code and Year. An inner join was selected to retain only the country-year combinations present in both randomly sampled datasets. We also used `select()` to avoid duplicating the country name variable. The resulting dataset contains 15,708 observations and five variables, representing 238 geographical records across 66 years (1960–2025). We used `dim()` to verify the merged structure.

```
# Import the World Bank country metadata
country_metadata <- read_csv("Metadata_Country_API_NY.GDP.PCAP.CD_DS2_en_csv_v2_1529.csv", show_col_types = FALSE) %>%
  select(Country_Code = `Country Code`, Region, IncomeGroup)
```

```
## New names:
## • `` -> `...6`
```

```
# Check metadata structure and missing Region values
dim(country_metadata)
```

```
## [1] 265   3
```

```
sum(is.na(country_metadata$Region))
```

```
## [1] 48
```

```
country_metadata %>% count(Country_Code) %>% filter(n > 1) %>% nrow()
```

```
## [1] 0
```

```
# Attach country information to the merged dataset
merged_with_metadata <- merged_data %>%
  left_join(country_metadata, by = "Country_Code")
# Keep individual countries and economies only
country_data <- merged_with_metadata %>%
  filter(!is.na(Region))
# Verify the country-level dataset
dim(country_data)
```

```
## [1] 12804     7
```

```
n_distinct(country_data$Country_Code)
```

```
## [1] 194
```

We used the accompanying World Bank country metadata as supporting information to identify individual countries and economies and distinguish them from regional aggregates. The metadata contains 265 records, including 48 with missing Region values, and no duplicate country codes. We then applied `left_join()` to attach Region and IncomeGroup classifications to the merged dataset. These classifications were used to identify the geographical categories of existing observations rather than introduce additional economic or health measurements. Since regional aggregates represent groups of countries rather than individual economies, we used `filter(!is.na(Region))` to exclude them and maintain a consistent country-level analysis.

The results showed that 44 regional and other aggregate records were excluded from the 238 matched geographical records. The resulting dataset contains 12,804 observations and seven variables, representing 194 individual countries and economies across 66 years (1960–2025). Importantly, the metadata did not modify, replace, or recalculate any GDP per capita or life expectancy values for the retained observations. Instead, it was used only to provide country classifications and support the exclusion of aggregates. Therefore, the analysis remains based on the two original World Bank indicator datasets, and their missing values were preserved for investigation during the scanning stage.

5. Understand

```
# Examine the dimensions and structure of the merged dataset
dim(country_data)
```

```
## [1] 12804     7
```

```
str(country_data)
```

```
## tibble [12,804 × 7] (S3: tbl_df/tbl/data.frame)
##  $ Country_Name   : chr [1:12804] "Myanmar" "Myanmar" "Myanmar" "Myanmar" ...
##  $ Country_Code   : chr [1:12804] "MMR" "MMR" "MMR" "MMR" ...
##  $ Year           : int [1:12804] 1960 1961 1962 1963 1964 1965 1966 1967 1968 1969 ...
##  $ GDP_per_capita : num [1:12804] 25.1 27.3 27.9 25.8 17.3 ...
##  $ Life_expectancy: num [1:12804] 44.1 44.6 45.3 46 46.7 ...
##  $ Region         : chr [1:12804] "East Asia & Pacific" "East Asia & Pacific" "East Asia & Pacific" "East Asia & Pacifi
c" ...
##  $ IncomeGroup    : chr [1:12804] "Lower middle income" "Lower middle income" "Lower middle income" "Lower middle incom
e" ...
```

```
# Check the class of each variable
sapply(country_data, class)
```

```
##    Country_Name    Country_Code            Year  GDP_per_capita Life_expectancy
##     "character"     "character"       "integer"       "numeric"       "numeric"
##          Region     IncomeGroup
##     "character"     "character"
```

```
# Examine the categorical variables
unique(country_data$Region)
```

```
## [1] "East Asia & Pacific"        "Sub-Saharan Africa"
## [3] "Europe & Central Asia"      "Latin America & Caribbean"
## [5] "Middle East & North Africa" "South Asia"
## [7] "North America"
```

```
unique(country_data$IncomeGroup)
```

```
## [1] "Lower middle income" "Low income"          "High income"
## [4] "Upper middle income"
```

```
# Check the frequency of income groups
table(country_data$IncomeGroup, useNA = "ifany")
```

```
##
##         High income          Low income Lower middle income Upper middle income
##                5016                1518                2772                3498
```

We used `dim()` and `str()` to examine the dimensions and structure of the country-level dataset. The results showed that the dataset is stored as a tibble containing 12,804 observations and seven variables. We then used `sapply()` with `class()` to identify the data type of each variable. Country_Name, Country_Code, Region, and IncomeGroup were classified as character variables, Year as integer, and GDP_per_capita and Life_expectancy as numeric. We also used `unique()` to examine the categorical values, identifying seven geographical regions and four income groups. The frequency table showed that high-income economies accounted for the largest number of country-year observations (5,016), while low-income economies accounted for the smallest (1,518). Based on these results, Region and IncomeGroup were identified as suitable variables for factor conversion.

```
# Convert Region to an unordered factor
# Convert IncomeGroup to an ordered and labelled factor
country_data <- country_data %>%
  mutate(
    Region = factor(Region),
    IncomeGroup = factor(
      IncomeGroup,
      levels = c(
        "Low income",
        "Lower middle income",
        "Upper middle income",
        "High income"
      ),
      labels = c(
        "Low income",
        "Lower-middle income",
        "Upper-middle income",
        "High income"
      ),
      ordered = TRUE
    )
  )
# Verify the data types after conversion
sapply(country_data, class)
```

```
## $Country_Name
## [1] "character"
##
## $Country_Code
## [1] "character"
##
## $Year
## [1] "integer"
##
## $GDP_per_capita
## [1] "numeric"
##
## $Life_expectancy
## [1] "numeric"
##
## $Region
## [1] "factor"
##
## $IncomeGroup
## [1] "ordered" "factor"
```

```
# Check factor levels and their ordering
levels(country_data$Region)
```

```
## [1] "East Asia & Pacific"        "Europe & Central Asia"
## [3] "Latin America & Caribbean"  "Middle East & North Africa"
## [5] "North America"              "South Asia"
## [7] "Sub-Saharan Africa"
```

```
levels(country_data$IncomeGroup)
```

```
## [1] "Low income"          "Lower-middle income" "Upper-middle income"
## [4] "High income"
```

```
is.ordered(country_data$IncomeGroup)
```

```
## [1] TRUE
```

We used `factor()` to convert Region from character to an unordered factor because geographical regions are categorical variables without a natural ranking. We also converted IncomeGroup into an ordered factor using four levels, ranging from Low income to High income. The `labels` argument was used to provide consistent and readable category names. We then used `sapply()` with `class()` to verify the conversions and `levels()` to examine the factor categories. The results confirmed that Region was successfully converted into a factor, while IncomeGroup was classified as an ordered factor. The `is.ordered()` function also returned TRUE, confirming the intended ordering. The dataset continues to contain character, numeric, integer, and factor variables, satisfying the required variety of data types.

6. Tidy & Manipulate Data I

Both World Bank datasets were originally presented in wide format, with each country or economy occupying one row and annual measurements from 1960 to 2025 stored in separate columns. We found that this structure is untidy for country-year analysis because the year values are embedded in column names rather than stored in a single variable. And to satisfy the tidy data principles, each year should be represented as a value in one Year column, with the corresponding GDP per capita or life expectancy measurement stored in another column. Therefore, we used `pivot_longer()` in Section 4 to reshape both sampled datasets into long format before merging them.

```
# Show the original wide format (three example years)
head(
  gdp_sample[, c("Country Name", "Country Code", "1960", "1961", "1962")], 3)
```

Country Name <chr>	Country Code <chr>	1960 <dbl>	1961 <dbl>	1962 <dbl>
Myanmar	MMR	25.08478	27.26545	27.9470
Malawi	MWI	*NA*	*NA*	*NA*
Oman	OMN	83.38531	84.53116	102.3259

3 rows

```
head(
  life_sample[, c("Country Name", "Country Code", "1960", "1961", "1962")],3)
```

Country Name <chr>	Country Code <chr>
Middle East, North Africa, Afghanistan & Pakistan (excluding high income)	MNA
Niger	NER
Cabo Verde	CPV

3 rows | 1-4 of 5 columns

```
# Show the tidy long format of both datasets
head(gdp_tidy, 3)
```

Country_Name <chr>	Country_Code <chr>	Year <int>	GDP_per_capita <dbl>
Myanmar	MMR	1960	25.08478
Myanmar	MMR	1961	27.26545
Myanmar	MMR	1962	27.94700

3 rows

```
head(life_tidy, 3)
```

Country_Name <chr>	Country_Code <chr>
Middle East, North Africa, Afghanistan & Pakistan (excluding high income)	MNA
Middle East, North Africa, Afghanistan & Pakistan (excluding high income)	MNA
Middle East, North Africa, Afghanistan & Pakistan (excluding high income)	MNA

3 rows

```
# Verify dimensions after reshaping
dim(gdp_tidy)
```

```
## [1] 16632     4
```

```
dim(life_tidy)
```

```
## [1] 16632     4
```

We used `head()` to compare the structure of the GDP dataset before and after tidying. The original samples contained 252 geographical records and 71 variables, with annual measurements stored in separate columns such as 1960, 1961, and 1962. After applying `pivot_longer()`, the results showed that both datasets contained 16,632 observations and four variables. The Year column now stores reporting years as integer values, while GDP_per_capita and Life_expectancy contain their corresponding numeric measurements. The reshaped datasets satisfy the three tidy data principles: each variable occupies one column, each country-year observation occupies one row, and each cell contains a single value. Missing indicator values were preserved for further investigation during the scanning stage. This structure also allows the two datasets to be merged consistently using Country_Code and Year.

7. Tidy & Manipulate Data II

```
# Create an ordered life expectancy group
country_data <- country_data %>%
  mutate(
    Life_Expectancy_Group = cut(
      Life_expectancy,
      breaks = c(-Inf, 60, 70, 80, Inf),
      labels = c("Below 60", "60-69", "70-79", "80+"),
      right = FALSE,
      ordered_result = TRUE
    )
  )
# Preview the new variable
head(
  country_data %>%
    select(Country_Name, Year, Life_expectancy, Life_Expectancy_Group), 5)
```

Country_Name <chr>	Year <int>	Life_expectancy <dbl>	Life_Expectancy_Group <ord>
Myanmar	1960	44.086	Below 60
Myanmar	1961	44.603	Below 60

Country_Name <chr>	Year <int>	Life_expectancy <dbl>	Life_Expectancy_Group <ord>
Myanmar	1962	45.264	Below 60
Myanmar	1963	45.951	Below 60
Myanmar	1964	46.653	Below 60

5 rows

```
# Check the structure and levels
str(country_data$Life_Expectancy_Group)
```

```
##  Ord.factor w/ 4 levels "Below 60"<"60-69"<..: 1 1 1 1 1 1 1 1 1 ...
```

```
levels(country_data$Life_Expectancy_Group)
```

```
## [1] "Below 60" "60-69"    "70-79"    "80+"
```

```
# Check the frequency of each group, including missing values
table(country_data$Life_Expectancy_Group, useNA = "ifany")
```

```
##
## Below 60    60-69    70-79      80+     <NA>
##     3511     3719     4549      797      228
```

```
# Verify the updated dimensions
dim(country_data)
```

```
## [1] 12804     8
```

We used `mutate()` and `cut()` to create a new variable called Life_Expectancy_Group from the existing Life_expectancy variable. The measurements were classified into four categories: Below 60, 60–69, 70–79, and 80+ years. These categories were created to simplify comparisons of life expectancy across countries and years. We set `right = FALSE` to ensure that boundary values were included in the appropriate categories and `ordered_result = TRUE` to create an ordered factor. The original Life_expectancy measurements were preserved. The results showed that the 70–79 group contained the largest number of observations (4,549), while the 80+ group contained the smallest (797). There were also 228 missing classifications, corresponding to missing life expectancy values. The dataset retained all 12,804 observations and now contains eight variables.

8. Scan I

```
# Create a summary table for all variables
missing_summary <- data.frame(
  Variable = names(country_data),
  Missing_Count = colSums(is.na(country_data)),
  Missing_Percent = round(
    colMeans(is.na(country_data)) * 100, 2
  )
)
missing_summary
```

	Variable <chr>	Missing_Count <dbl>	Missing_Percent <dbl>
Country_Name	Country_Name	0	0.00
Country_Code	Country_Code	0	0.00
Year	Year	0	0.00
GDP_per_capita	GDP_per_capita	2304	17.99
Life_expectancy	Life_expectancy	228	1.78
Region	Region	0	0.00
IncomeGroup	IncomeGroup	0	0.00

	Variable <chr>	Missing_Count <dbl>	Missing_Percent <dbl>
Life_Expectancy_Group	Life_Expectancy_Group	228	1.78

8 rows

```
# Count observations containing at least one missing value
sum(!complete.cases(country_data))
```

```
## [1] 2475
```

We used `is.na()` with `colSums()` and `colMeans()` to identify the number and percentage of missing values in each variable. The results showed that GDP_per_capita contained 2,304 missing values (17.99%), while Life_expectancy contained 228 missing values (1.78%). The derived Life_Expectancy_Group variable also contained 228 missing values because its categories depend directly on Life_expectancy. The remaining five variables had no missing values. GDP per capita therefore showed the highest proportion of missing observations. Further investigation was required to determine whether these missing values were concentrated in particular years or countries before selecting an appropriate treatment method.

```
# Examine missing values by year
missing_by_year <- country_data %>%
  group_by(Year) %>%
  summarise(
    GDP_missing = sum(is.na(GDP_per_capita)),
    Life_missing = sum(is.na(Life_expectancy)),
    .groups = "drop"
  )
# Years with the most missing GDP observations
head(
  missing_by_year %>% arrange(desc(GDP_missing)),
  5
)
```

Year <int>	GDP_missing <int>	Life_missing <int>
1960	95	2
1961	93	1
1962	91	1
1963	91	2
1964	91	2

5 rows

```
# Years with the most missing life expectancy observations
head(
  missing_by_year %>% arrange(desc(Life_missing)),
  5
)
```

Year <int>	GDP_missing <int>	Life_missing <int>
2025	27	194
1960	95	2
1963	91	2
1964	91	2
1965	87	2

5 rows

```
# Identify countries with the most missing GDP observations
missing_by_country <- country_data %>%
  group_by(Country_Code, Country_Name) %>%
  summarise(
    GDP_missing = sum(is.na(GDP_per_capita)),
    Life_missing = sum(is.na(Life_expectancy)),
    .groups = "drop"
  )
head(
  missing_by_country %>% arrange(desc(GDP_missing)),
  5
)
```

Country_Code <chr>	Country_Name <chr>	GDP_missing <int>	Life_missing <int>
GIB	Gibraltar	66	1
PRK	Korea, Dem. People's Rep.	66	1
MAF	St. Martin (French part)	62	1
SSD	South Sudan	58	1
SXM	Sint Maarten (Dutch part)	49	1

5 rows

```
# Check how many rows contain at least one missing value
sum(!complete.cases(country_data))
```

```
## [1] 2475
```

We used `group_by()` and `summarise()` to investigate missing-value patterns across years and countries. The results showed that GDP per capita missingness was particularly high in earlier years, with 95 missing observations in 1960 and 93 in 1961. For life expectancy, all 194 observations in 2025 were missing, indicating that no life expectancy measurements were available for that year in the sampled country-level dataset. We also identified countries with substantial missing GDP information. Gibraltar and North Korea had missing GDP per capita values across all 66 years, while St. Martin (French part) and South Sudan had 62 and 58 missing observations, respectively. Finally, `complete.cases()` showed that 2,475 observations (19.33%) contained at least one missing value. These results indicate that missingness varies considerably across countries and years. Therefore, applying a single overall mean to replace missing values would be inappropriate, as it could distort historical and cross-country differences.

```
# Check for duplicate country-year observations
country_data %>% count(Country_Code, Year) %>% filter(n > 1) %>% nrow()
```

```
## [1] 0
```

```
# Check for empty country identifiers
sum(str_trim(country_data$Country_Name) == "", na.rm = TRUE)
```

```
## [1] 0
```

```
sum(str_trim(country_data$Country_Code) == "", na.rm = TRUE)
```

```
## [1] 0
```

```
# Check for invalid reporting years
sum(!country_data$Year %in% 1960:2025)
```

```
## [1] 0
```

```
# Check for invalid or unusual numeric values
sum(country_data$GDP_per_capita <= 0, na.rm = TRUE)
```

```
## [1] 0
```

```
sum(country_data$Life_expectancy <= 0, na.rm = TRUE)
```

```
## [1] 0
```

```
sum(country_data$Life_expectancy > 120, na.rm = TRUE)
```

```
## [1] 0
```

```
# Check special numeric values
sum(is.infinite(country_data$GDP_per_capita))
```

```
## [1] 0
```

```
sum(is.infinite(country_data$Life_expectancy))
```

```
## [1] 0
```

```
sum(is.nan(country_data$GDP_per_capita))
```

```
## [1] 0
```

```
sum(is.nan(country_data$Life_expectancy))
```

```
## [1] 0
```

```
# Confirm that missing derived categories match missing life expectancy
sum(
  is.na(country_data$Life_Expectancy_Group) !=
  is.na(country_data$Life_expectancy)
)
```

```
## [1] 0
```

We used count() and filter() to check for repeated country-year records and str_trim() to identify empty country identifiers. We also examined reporting years outside the expected range of 1960–2025 and checked for non-positive GDP per capita, non-positive life expectancy, and life expectancy values exceeding 120 years. In addition, `is.infinite()` and `is.nan()` were used to identify special numeric values. We also verified that missing values in Life_Expectancy_Group corresponded to missing Life_expectancy measurements. All 12 checks returned zero, indicating that no observations violated the specified validation rules. However, these results do not rule out other potential inconsistencies, particularly unusually large changes between consecutive years. Further examination was therefore required.

```
# Calculate annual changes in life expectancy
life_changes <- country_data %>%
  arrange(Country_Code, Year) %>%
  group_by(Country_Code, Country_Name) %>%
  mutate(
    Annual_Change = Life_expectancy - lag(Life_expectancy)
  ) %>%
  ungroup()
# Identify changes greater than 10 years in either direction
suspicious_changes <- life_changes %>%
  filter(
    !is.na(Annual_Change),
    abs(Annual_Change) > 10
  ) %>%
  arrange(desc(abs(Annual_Change))) %>%
  select(
    Country_Name, Year,
    Life_expectancy, Annual_Change
  )
# Count and examine suspicious changes
nrow(suspicious_changes)
```

```
## [1] 47
```

```
head(suspicious_changes, 5)
```

Country_Name <chr>	Year <int>	Life_expectancy <dbl>	Annual_Change <dbl>
Central African Republic	2023	57.408	38.590
Central African Republic	2009	14.665	-35.208
Central African Republic	2010	46.871	32.206
Cambodia	1979	41.072	29.499
Cambodia	1975	12.784	-26.512

5 rows

```
# Examine suspicious changes in surrounding years
life_changes %>%
  filter(
    Country_Name == "Central African Republic",
    Year %in% c(2007:2011, 2021:2024)
  ) %>%
  select(
    Country_Name, Year,
    Life_expectancy, Annual_Change
  )
```

Country_Name <chr>	Year <int>	Life_expectancy <dbl>	Annual_Change <dbl>
Central African Republic	2007	49.327	0.640
Central African Republic	2008	49.873	0.546
Central African Republic	2009	14.665	-35.208
Central African Republic	2010	46.871	32.206
Central African Republic	2011	50.107	3.236
Central African Republic	2021	40.279	-10.317
Central African Republic	2022	18.818	-21.461
Central African Republic	2023	57.408	38.590
Central African Republic	2024	57.671	0.263

9 rows

```
# Check for unusually low life expectancy measurements
country_data %>%
  filter(
    !is.na(Life_expectancy),
    Life_expectancy < 20
  ) %>%
  arrange(Life_expectancy) %>%
  select(
    Country_Name, Year, Life_expectancy
  ) %>%
  head(10)
```

Country_Name <chr>	Year <int>	Life_expectancy <dbl>
South Sudan	1988	10.989
Cambodia	1977	11.295
Cambodia	1978	11.573

Country_Name <chr>	Year <int>	Life_expectancy <dbl>
Cambodia	1976	11.632
Cambodia	1975	12.784
Central African Republic	2009	14.665
Central African Republic	2022	18.818
South Sudan	1998	19.048

8 rows

We used `lag()` to calculate annual changes in life expectancy for each country after arranging observations by country and year. Changes exceeding 10 years in either direction were flagged for further investigation, as such large annual differences may indicate potential inconsistencies. The results identified 47 observations exceeding this threshold. The largest increase occurred in the Central African Republic in 2023 (+38.590 years), while the largest decrease occurred in the same country in 2009 (−35.208 years). Cambodia also showed substantial changes in 1975 and 1979. These observations were considered potential inconsistencies rather than confirmed errors. Large changes may reflect historical events or data estimation issues. Therefore, further examination of surrounding years and source information was necessary before deciding whether to retain or modify the values.

We further examined the surrounding years for the Central African Republic and identified unusually large fluctuations. Life expectancy decreased from 49.873 years in 2008 to 14.665 in 2009 before increasing to 46.871 in 2010. A similar pattern occurred between 2021 and 2023, with life expectancy falling to 18.818 years in 2022 before increasing to 57.408 in 2023. We also identified eight observations with life expectancy below 20 years, involving Cambodia, South Sudan, and the Central African Republic. A comparison with published World Bank data confirmed that these unusual values were present in the source series. Historical events, including the Khmer Rouge period in Cambodia, may help explain some extreme mortality estimates. Since the checks did not establish that these observations were incorrect, we retained their original values rather than introducing unsupported corrections. These observations will be examined further during outlier detection, and their potential influence will be acknowledged as a limitation.

```
# Create an analysis dataset without 2025
# Life expectancy is unavailable for all countries in 2025
analysis_data <- country_data %>%
  filter(Year != 2025)
# Verify the dimensions
dim(country_data)
```

```
## [1] 12804     8
```

```
dim(analysis_data)
```

```
## [1] 12610     8
```

```
# Check remaining missing indicator values
colSums(is.na(
  analysis_data[, c(
    "GDP_per_capita",
    "Life_expectancy",
    "Life_Expectancy_Group"
  )]
))
```

```
##        GDP_per_capita       Life_expectancy Life_Expectancy_Group
##                  2277                    34                    34
```

```
# Check the remaining incomplete observations
sum(!complete.cases(analysis_data))
```

```
## [1] 2281
```

After investigating the missing-value patterns, we found that all 194 country-level life expectancy observations for 2025 were missing. Since these observations could not contribute to an analysis involving both GDP per capita and life expectancy, we used `filter()` to exclude 2025 from the analysis dataset. We created a separate dataset called analysis_data to preserve the original

country_data for comparison. The results showed that the number of observations decreased from 12,804 to 12,610, while all eight variables were retained. After filtering, GDP_per_capita contained 2,277 missing values, while Life_expectancy and its derived category each contained 34 missing values. A total of 2,281 observations still contained at least one missing value.

```
# Count the GDP values available for each country
no_gdp <- analysis_data %>%
  group_by(Country_Code, Country_Name) %>%
  summarise(GDP_values = sum(!is.na(GDP_per_capita)), .groups = "drop") %>%
  filter(GDP_values == 0)
# Remove these countries with no gdp
no_gdp
```

Country_Code <chr>	Country_Name <chr>	GDP_values <int>
GIB	Gibraltar	0
PRK	Korea, Dem. People's Rep.	0

2 rows

```
analysis_data <- analysis_data %>%
  filter(!Country_Code %in% no_gdp$Country_Code)
dim(analysis_data)
```

```
## [1] 12480     8
```

```
# Remove the remaining observations with missing values
analysis_data <- analysis_data[complete.cases(analysis_data), ]
# Check the result
dim(analysis_data)
```

```
## [1] 10329     8
```

```
n_distinct(analysis_data$Country_Code)
```

```
## [1] 192
```

```
colSums(is.na(analysis_data))
```

```
##          Country_Name          Country_Code                  Year
##                     0                     0                     0
##        GDP_per_capita       Life_expectancy                Region
##                     0                     0                     0
##           IncomeGroup Life_Expectancy_Group
##                     0                     0
```

We first used `group_by()` and `summarise()` to count the GDP per capita values available for each country, which found two countries, Gibraltar and North Korea (listed as “Korea, Dem. People’s Rep.”), had no GDP values in any year. With no GDP information at all, these countries could not contribute to the analysis, so we removed them using `filter()`, which reduced the dataset to 12,480 observations from 192 countries. We then used `complete.cases()` to remove the remaining 2,151 observations that contained a missing value: 2,147 with missing GDP per capita and 4 with missing life expectancy only. Only GDP per capita, life expectancy and the life expectancy group contained missing values, so no observations were removed because of other variables. We chose deletion rather than imputation for three reasons:

- Most missing GDP values are in early years, especially the 1960s, or in periods when a country did not yet exist as an independent economy or was not reporting data.
- Filling around 17% of GDP values with averages or medians would create many identical, made-up values and would disfigure the outlier checks in Section 9 and the transformation in Section 10.
- Our analysis compares GDP per capita with life expectancy, so each observation needs both values.

Our deletion removed about 17% of the observations as the missing values are concentrated in early years and in some lower-income countries, the cleaned data under-represents these groups, which is a limitation we found. The final dataset contains 10,329 complete observations from 192 countries, with no missing values remaining.

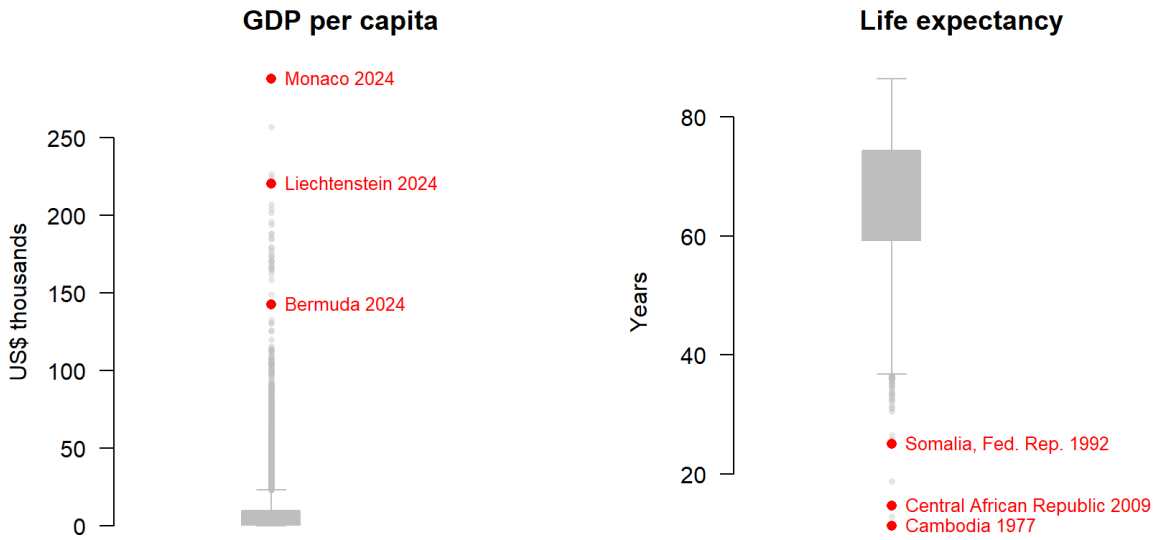
9. Scan II

```
# Year is a time index rather than a measurement, so it is not checked
outlier_vars <- c("GDP_per_capita", "Life_expectancy")
# Tukey's method: flag values below Q1 - 1.5*IQR or above Q3 + 1.5*IQR
outlier_summary <- bind_rows(lapply(outlier_vars, function(var) {
  x <- analysis_data[[var]]
  Q1 <- as.numeric(quantile(x, 0.25))
  Q3 <- as.numeric(quantile(x, 0.75))
  lower <- Q1 - 1.5 * IQR(x)
  upper <- Q3 + 1.5 * IQR(x)
  data.frame(Variable = var,
             Lower_Fence = round(lower, 2),
             Upper_Fence = round(upper, 2),
             Below_Lower = sum(x < lower),
             Above_Upper = sum(x > upper))
}))
outlier_summary
```

Variable <chr>	Lower_Fence <dbl>	Upper_Fence <dbl>	Below_Lower <int>	Above_Upper <int>
GDP_per_capita	-12935.55	23170.20	0	1356
Life_expectancy	36.69	96.88	51	0

2 rows

```
# Most extreme value for the three most extreme countries (used as labels)
top_gdp <- analysis_data %>%
  group_by(Country_Name) %>%
  filter(GDP_per_capita == max(GDP_per_capita)) %>%
  ungroup() %>%
  arrange(desc(GDP_per_capita)) %>%
  head(3)
low_life <- analysis_data %>%
  group_by(Country_Name) %>%
  filter(Life_expectancy == min(Life_expectancy)) %>%
  ungroup() %>%
  arrange(Life_expectancy) %>%
  head(3)
# Boxplots with the labelled points
par(mfrow = c(1, 2), mar = c(2, 5, 3, 1), las = 1)
boxplot(analysis_data$GDP_per_capita / 1000,
        main = "GDP per capita", ylab = "US$ thousands",
        col = "grey", border = "grey", boxwex = 0.5,
        whisklty = 1, frame.plot = FALSE, xlim = c(0.4, 2.2),
        outpch = 16, outcex = 0.6,
        outcol = adjustcolor("grey", alpha.f = 0.35))
points(rep(1, 3), top_gdp$GDP_per_capita / 1000, pch = 16, col = "red")
text(1, top_gdp$GDP_per_capita / 1000,
     labels = paste(top_gdp$Country_Name, top_gdp$Year),
     pos = 4, cex = 0.8, col = "red")
boxplot(analysis_data$Life_expectancy,
        main = "Life expectancy", ylab = "Years",
        col = "grey", border = "grey", boxwex = 0.5,
        whisklty = 1, frame.plot = FALSE, xlim = c(0.4, 2.2),
        outpch = 16, outcex = 0.6,
        outcol = adjustcolor("grey", alpha.f = 0.35))
points(rep(1, 3), low_life$Life_expectancy, pch = 16, col = "red")
text(1, low_life$Life_expectancy,
     labels = paste(low_life$Country_Name, low_life$Year),
     pos = 4, cex = 0.8, col = "red")
```




```
par(mfrow = c(1, 1))
```

We scanned the two numeric measurements, GDP_per_capita and Life_expectancy, for univariate outliers, however, Year was excluded because it is a time index rather than a measurement, and every value from 1960 to 2024 is already valid. We first used Tukey's boxplot method, which flags values below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR and then drew the boxplots with boxplot() and labelled the most extreme value for the three most extreme countries in each variable. GDP per capita is shown in thousands of US dollars for easier readability. GDP per capita had 1,356 outliers (13.1% of observations), all above the upper fence of $23,170.20, with Monaco, Liechtenstein and Bermuda the most extreme. Life expectancy had 51 outliers (0.5%), all below the lower fence of 36.69 years, with the lowest values in Cambodia (1975–1978, lowest 11.3 years in 1977), the Central African Republic (2009) and Somalia (1992).

```
# z-scores (outliers package): flag values more than 3 standard deviations from the mean
z_gdp  <- analysis_data$GDP_per_capita %>% scores(type = "z")
z_life <- analysis_data$Life_expectancy %>% scores(type = "z")
length(which(abs(z_gdp) > 3))
```

```
## [1] 210
```

```
length(which(abs(z_life) > 3))
```

```
## [1] 19
```

```
# Countries behind the flagged values
n_distinct(analysis_data$Country_Name[which(abs(z_gdp) > 3)])
```

```
## [1] 17
```

```
table(analysis_data$Country_Name[which(abs(z_life) > 3)])
```

```
##
##              Bangladesh                 Burundi                Cambodia
##                       1                       1                       4
## Central African Republic        Equatorial Guinea                    Mali
##                       3                       3                       3
##            Sierra Leone       Somalia, Fed. Rep.
##                       1                       3
```

Next, we used the z-score method with `scores()` from the outliers package, which flags values more than three standard deviations from the mean. This identified 210 GDP per capita values from 17 countries and 19 life expectancy values from 8 countries. The z-score method flagged fewer values than Tukey's method as it assumes a roughly normal distribution, while the boxplots show that GDP per capita is strongly right-skewed. Therefore, we relied mainly on Tukey's method, which does not require normality. (Both methods identified the same extreme cases, such as Monaco, Liechtenstein, Cambodia (1975–1978) and Somalia (1991–1992)).

```
# Fences for each variable
gdp_fence  <- quantile(analysis_data$GDP_per_capita, 0.75) +
```

```
  1.5 * IQR(analysis_data$GDP_per_capita)
life_fence <- quantile(analysis_data$Life_expectancy, 0.25) -
  1.5 * IQR(analysis_data$Life_expectancy)
# GDP outliers by income group and decade
gdp_outliers <- analysis_data %>% filter(GDP_per_capita > gdp_fence)
table(gdp_outliers$IncomeGroup)
```

```
##
##          Low income Lower-middle income Upper-middle income         High income
##                   0                   0                   0                1356
```

```
table(floor(gdp_outliers$Year / 10) * 10)
```

```
##
## 1970 1980 1990 2000 2010 2020
##   13   47  182  371  484  259
```

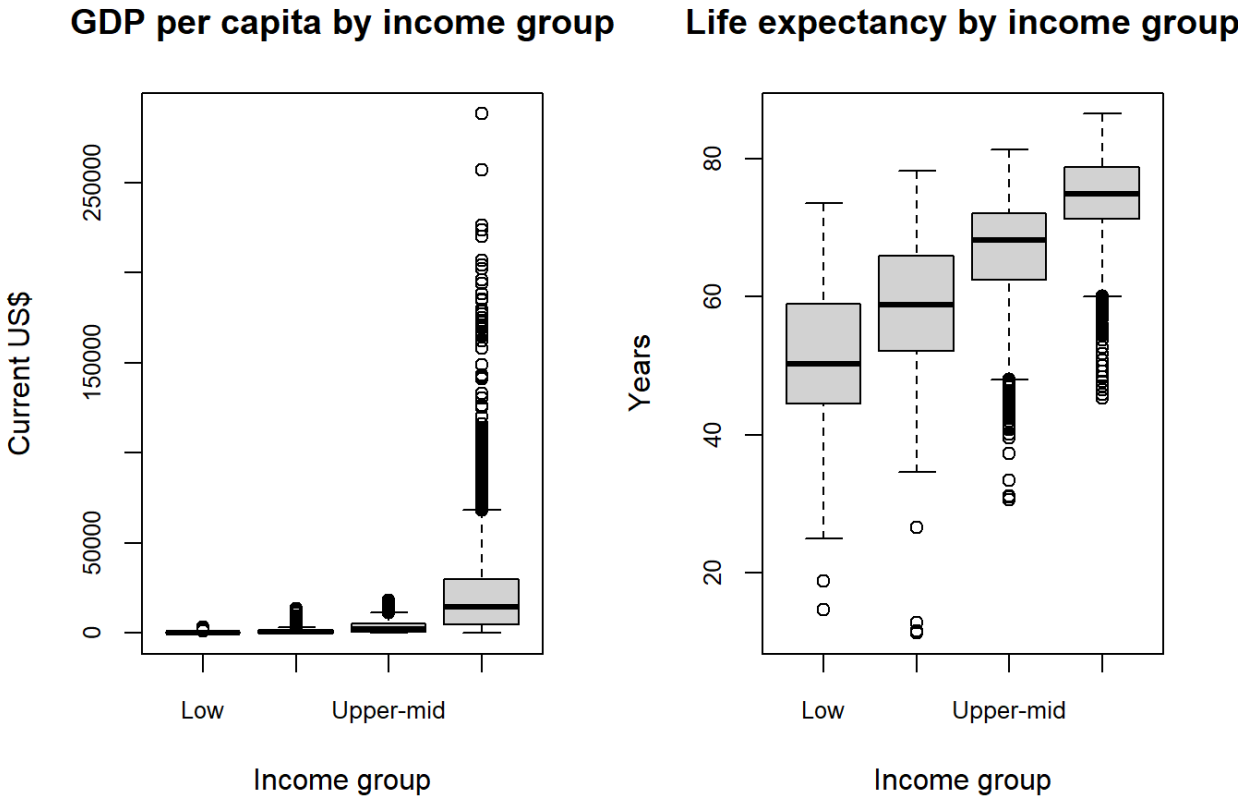
```
# Life expectancy outliers by country
analysis_data %>%
  filter(Life_expectancy < life_fence) %>%
  count(Country_Name, sort = TRUE)
```

Country_Name <chr>	**n** <int>
Niger	16
Mali	9
Sierra Leone	7
Cambodia	4
Central African Republic	3
Equatorial Guinea	3
Somalia, Fed. Rep.	3
Cameroon	2
Bangladesh	1
Burundi	1

1-10 of 12 rows Previous **1** 2 Next

We then examined which observations were flagged, to decide whether they were errors or genuine values. All 1,356 GDP outliers belong to high-income economies, and about 82% were recorded from 2000 onwards. We believe that this is because GDP per capita is measured in current US dollars, which are not adjusted for inflation, so values naturally rise over time. The 51 life expectancy outliers come from 12 countries with most coming from very low-income countries in the 1960s and 1970s, such as Niger (16), Mali (9) and Sierra Leone (7), when life expectancy was much lower than it is today. Others match well-known wars, famines or mass killings, such as Cambodia under the Khmer Rouge (1975–1978), Bangladesh's war of independence (1971), and famines in Ethiopia (1983) and Somalia (1991–1992 and 2011). The three Central African Republic values (2009, 2019 and 2022) are sudden one-year falls similar to those identified in Scan I.

```
# Bivariate boxplots: each variable by income group
par(mfrow = c(1, 2))
bp_gdp <- boxplot(GDP_per_capita ~ IncomeGroup, data = analysis_data,
                  names = c("Low", "Lower-mid", "Upper-mid", "High"),
                  main = "GDP per capita by income group",
                  xlab = "Income group", ylab = "Current US$", cex.axis = 0.8)
bp_life <- boxplot(Life_expectancy ~ IncomeGroup, data = analysis_data,
                   names = c("Low", "Lower-mid", "Upper-mid", "High"),
                   main = "Life expectancy by income group",
                   xlab = "Income group", ylab = "Years", cex.axis = 0.8)
```




```
par(mfrow = c(1, 1))
# Number of outliers within each income group
table(factor(bp_gdp$group, levels = 1:4, labels = bp_gdp$names))
```

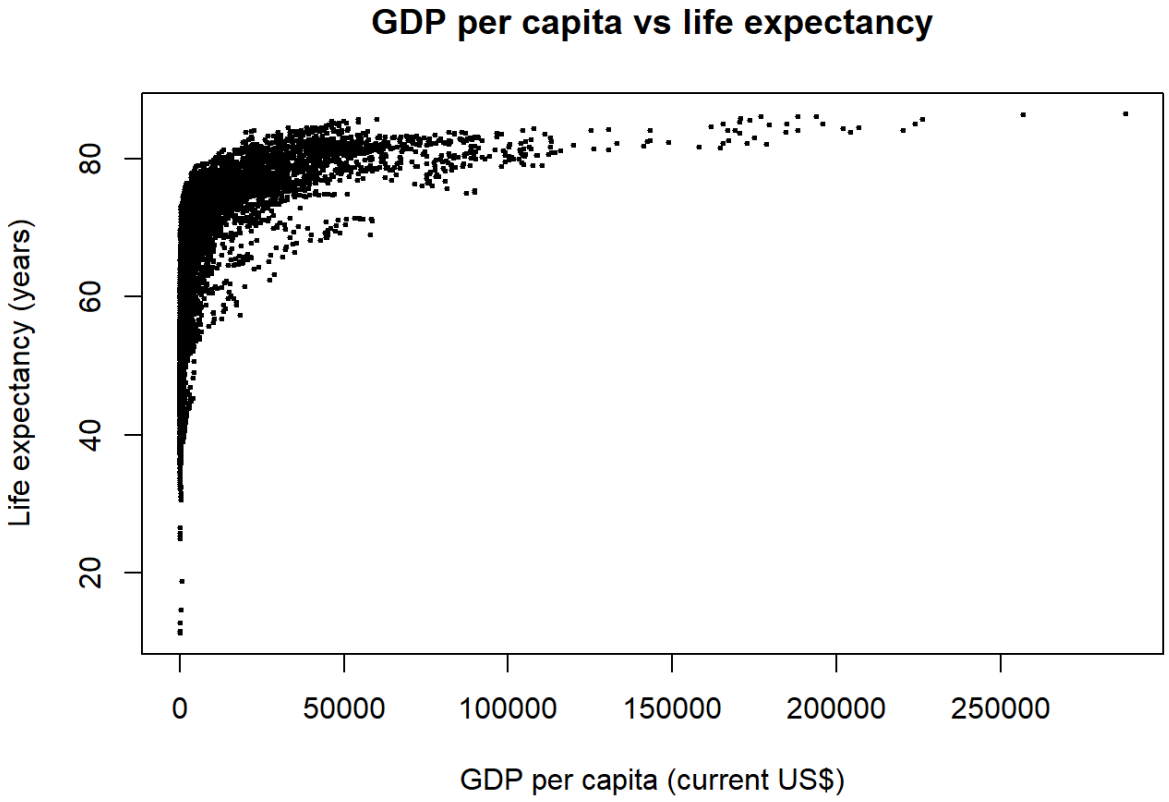
```
##
##       Low Lower-mid Upper-mid      High
##        54       190        75       207
```

```
table(factor(bp_life$group, levels = 1:4, labels = bp_life$names))
```

```
##
##       Low Lower-mid Upper-mid      High
##         2         5        56        91
```

To check for outliers within groups, we drew bivariate boxplots of each variable by income group. These flagged 526 GDP values and 154 life expectancy values as unusual within their own income group. Many of these reflect changes over time, because each country's current income group is applied to all of its years. For example, Syria is now classified as low income, but its GDP per capita before the civil war (1979–2012) was well above that of most low-income countries. Similarly, countries now classified as high or upper-middle income, such as Saudi Arabia, South Korea and Algeria, had much lower life expectancy in the 1960s and 1970s.

```
# Scatter plot of the two numeric variables
plot(analysis_data$GDP_per_capita, analysis_data$Life_expectancy,
     xlab = "GDP per capita (current US$)", ylab = "Life expectancy (years)",
     main = "GDP per capita vs life expectancy", pch = 20, cex = 0.5)
```




```
# Robust Mahalanobis distance (MVN package)
# Doornik-Hansen test is used because the default test needs too much
# memory for more than 10,000 observations
set.seed(4280335)
mv_results <- mvn(
  data = as.data.frame(analysis_data[, c("GDP_per_capita", "Life_expectancy")]),
  mvn_test = "doornik_hansen",
  multivariate_outlier_method = "quan",
  show_new_data = TRUE
)
# Multivariate normality test
mv_results$multivariate_normality
```

Test	Statistic	df	p.value	Method	MVN
<chr>	<dbl>	<dbl>	<chr>	<chr>	<chr>
Doornik-Hansen	29049.46	4	<0.001	asymptotic	✗ Not normal

1 row

```
# Number of potential multivariate outliers
mv_out <- mv_results$multivariate_outliers
nrow(mv_out)
```

```
## [1] 4225
```

```
# The five most extreme observations
mv_out <- mv_out %>% arrange(desc(Mahalanobis.Distance))
top_mv <- as.numeric(head(mv_out$Observation, 5))
analysis_data[top_mv, c("Country_Name", "Year", "GDP_per_capita", "Life_expectancy")]
```

Country_Name	Year	GDP_per_capita	Life_expectancy
<chr>	<int>	<dbl>	<dbl>
Monaco	2024	288001.6	86.497
Monaco	2023	256799.9	86.372

Country_Name <chr>	Year <int>	GDP_per_capita <dbl>	Life_expectancy <dbl>
Monaco	2022	226052.1	85.746
Monaco	2021	223823.3	85.113
Liechtenstein	2024	220167.2	84.200

5 rows

```
# Size of the data if all multivariate outliers were removed
dim(mv_results$new_data)
```

```
## [1] 6104    2
```

To look for potential multivariate outliers, we first plotted GDP per capita against life expectancy. The scatter plot shows a curved relationship: life expectancy rises quickly as GDP per capita increases from low levels and then levels off, while a small number of very wealthy economies stretch far to the right. We then used `mvn()` from the MVN package with `multivariate_outlier_method = "quan"`. This calculates robust Mahalanobis distances, which measure how unusual an observation is when both variables are considered together, and flags observations above the chi-square critical value at the 97.5% level. We then set `show_new_data = TRUE` to see how much data would remain if the flagged observations were removed.

A choice we made was using the Doornik–Hansen test (Doornik and Hansen 2008) (`mvn_test = "doornik_hansen"`) instead of the default Henze–Zirkler test as the default test needs too much memory for more than 10,000 observations. In the end, it showed that the two variables are not multivariate normal ($p < 0.001$). The method flagged 4,225 observations (40.9% of the data) as potential multivariate outliers, with Monaco and Liechtenstein the most extreme. This very large number is mainly a result of the strong skewness in GDP per capita. The robust method centres on the most common values, which are low-income observations, so most observations from middle- and high-income economies look unusual. Removing all flagged observations would leave only 6,104 of the 10,329 observations.

```
# Capping function from Module 6
cap <- function(x){
  quantiles <- quantile(x, c(.05, 0.25, 0.75, .95))
  x[x < quantiles[2] - 1.5*IQR(x)] <- quantiles[1]
  x[x > quantiles[3] + 1.5*IQR(x)] <- quantiles[4]
  x
}
# Apply capping to copies of the variables (original data unchanged)
gdp_capped  <- analysis_data$GDP_per_capita %>% cap()
life_capped <- analysis_data$Life_expectancy %>% cap()
# Compare original and capped values
summary(analysis_data$GDP_per_capita)
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
##     11.8    604.1   2202.3   9797.3   9630.5 288001.6
```

```
summary(gdp_capped)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    11.8   604.1  2202.3  9518.8  9630.5 46572.7
```

```
summary(analysis_data$Life_expectancy)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   11.29   59.26   68.59   66.11   74.31   86.50
```

```
summary(life_capped)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   36.74   59.26   68.59   66.18   74.31   86.50
```

```
# Number of values that capping would increase or decrease
sum(gdp_capped > analysis_data$GDP_per_capita)
```

```
## [1] 839
```

```
sum(gdp_capped < analysis_data$GDP_per_capita)
```

```
## [1] 517
```

```
sum(life_capped > analysis_data$Life_expectancy)
```

```
## [1] 51
```

Finally, we tested capping (winsorising) using the `cap()` function on copies of the two variables, so the original data was not changed. For GDP per capita, capping replaced all 1,356 outliers with the 95th percentile ($46,573), reducing the maximum from $288,002. However, because 13.1% of values were above the upper fence, 839 of them were actually increased, as they lay between the fence ($23,170) and the 95th percentile. For life expectancy, capping raised all 51 outliers to the 5th percentile (45.1 years). As a result, Cambodia's value of 11.3 years in 1977 would end up higher than 466 observations that were not outliers.

Decision on outliers

After investigating the outliers and testing the Module 6 handling methods, we kept all observations in their original form:

- The outliers are genuine values, not data entry or processing errors and they reflect real differences in wealth, low life expectancy in the 1960s and 1970s, and real wars and famines. Module 6 recommends removing or imputing outliers mainly when they result from errors.
- Removing outliers would remove a large share of the data: 13.1% of observations under Tukey's method and 40.9% under the MVN method, including most recent observations from high-income economies.
- Capping disfigured both variables as it increased 839 GDP values, gave all 1,356 GDP outliers the same value, and placed the lowest life expectancy values above 466 ordinary observations.

Instead, we will reduce the influence of the extreme GDP values with a log transformation later in #10 Transform, which is another way to handle outliers. Additionally, the unusual Central African Republic values remain a possible data-quality issue and a limitation of this analysis, therefore the dataset still contains 10,329 observations from 192 countries.

10. Transform

```
# A log cannot be applied to zero or negative values, so we first checked the minimum
min(analysis_data$GDP_per_capita)
```

```
## [1] 11.80132
```

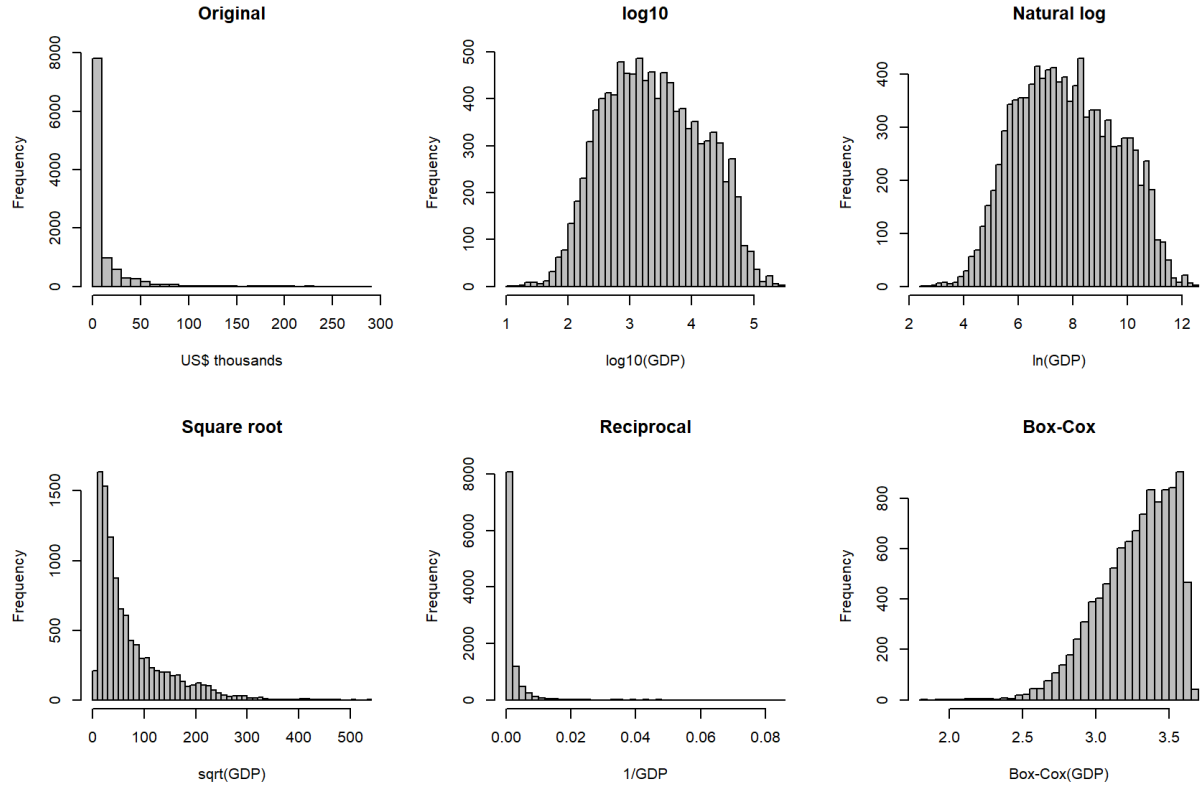
```
# Different transformations on GDP per capita
gdp <- analysis_data$GDP_per_capita
gdp_log10 <- log10(gdp)
gdp_ln <- log(gdp)
gdp_sqrt <- sqrt(gdp)
gdp_recip <- 1 / gdp
gdp_boxcox <- BoxCox(gdp, lambda = "auto")
attr(gdp_boxcox, "lambda")
```

```
## [1] -0.2609916
```

```
# Skewness of each version
skew_value <- function(x) mean((x - mean(x))^3) / mean((x - mean(x))^2)^1.5
round(c(Original = skew_value(gdp), Log10 = skew_value(gdp_log10),
        Ln = skew_value(gdp_ln), Sqrt = skew_value(gdp_sqrt),
        Reciprocal = skew_value(gdp_recip), BoxCox = skew_value(gdp_boxcox)), 2)
```

```
##   Original      Log10         Ln       Sqrt Reciprocal     BoxCox
##       4.48       0.10       0.10       1.78       7.99      -0.81
```

```
# Histograms of each version
par(mfrow = c(2, 3))
hist(gdp / 1000, breaks = 40, col = "grey", main = "Original", xlab = "US$ thousands")
hist(gdp_log10,  breaks = 40, col = "grey", main = "log10", xlab = "log10(GDP)")
hist(gdp_ln,     breaks = 40, col = "grey", main = "Natural log", xlab = "ln(GDP)")
hist(gdp_sqrt,   breaks = 40, col = "grey", main = "Square root", xlab = "sqrt(GDP)")
hist(gdp_recip,  breaks = 40, col = "grey", main = "Reciprocal", xlab = "1/GDP")
hist(gdp_boxcox, breaks = 40, col = "grey", main = "Box-Cox", xlab = "Box-Cox(GDP)")
```



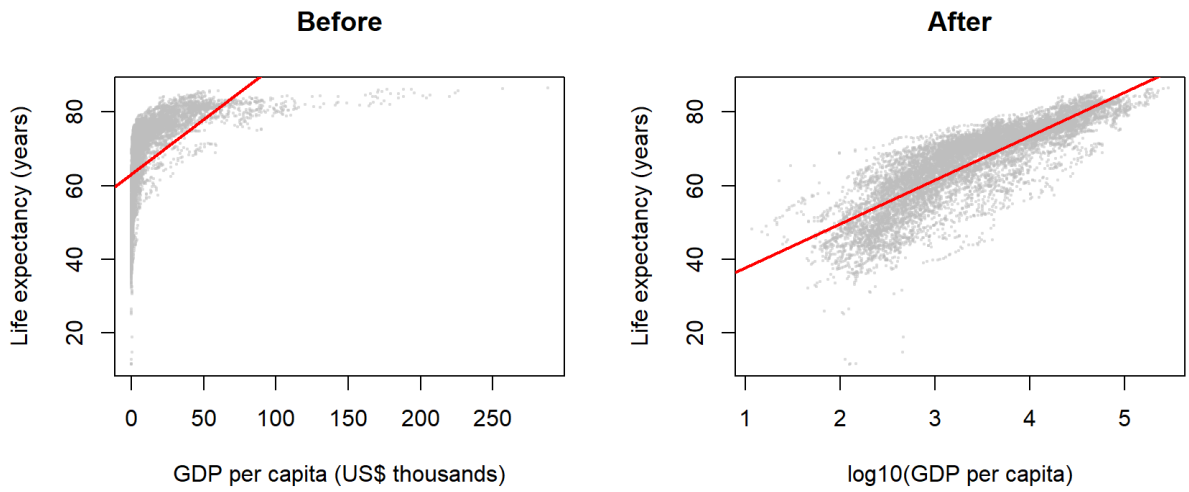

```
par(mfrow = c(1, 1))
```

First, we tried several transformations on GDP per capita and compared their histograms and skewness, so that we could keep the one that worked best. However, we needed to make sure that all GDP values are positive so every transformation could be applied directly. We then found that the original GDP values are strongly right-skewed (skewness 4.48). In class, we were advised to use roots, logarithms or reciprocals for right-skewed data, so we tested log10, the natural log, the square root and the reciprocal. We also tested a Box-Cox transformation, using BoxCox() from the forecast package with lambda = “auto”. The results were:

- The square root only partly reduced the skewness (1.78).
- The reciprocal however, made it much worse (7.99) as the smallest GDP values became the largest numbers.
- Box-Cox chose a lambda of about −0.26 and over-corrected, which left the data left-skewed (about −0.81).
- log10 and the natural log both produced a nearly symmetric shape (0.10).

We therefore chose log10, which also has the easiest scale to read: 3 means $1,000, 4 means $10,000 and 5 means $100,000.

```
# Keep the log10 version as a new variable
analysis_data <- analysis_data %>%
  mutate(log_GDP = log10(GDP_per_capita))
# Relationship with life expectancy before and after (with a straight line)
par(mfrow = c(1, 2))
plot(analysis_data$GDP_per_capita / 1000, analysis_data$Life_expectancy,
     pch = 20, cex = 0.4, col = adjustcolor("grey", alpha.f = 0.3),
     main = "Before", xlab = "GDP per capita (US$ thousands)",
     ylab = "Life expectancy (years)")
abline(lm(Life_expectancy ~ I(GDP_per_capita / 1000), data = analysis_data),
       col = "red", lwd = 2)
plot(analysis_data$log_GDP, analysis_data$Life_expectancy,
     pch = 20, cex = 0.4, col = adjustcolor("grey", alpha.f = 0.3),
     main = "After", xlab = "log10(GDP per capita)",
     ylab = "Life expectancy (years)")
abline(lm(Life_expectancy ~ log_GDP, data = analysis_data),
       col = "red", lwd = 2)
```




```
par(mfrow = c(1, 1))
# Correlation with life expectancy before and after
round(cor(analysis_data$GDP_per_capita, analysis_data$Life_expectancy), 3)
```

```
## [1] 0.527
```

```
round(cor(analysis_data$log_GDP, analysis_data$Life_expectancy), 3)
```

```
## [1] 0.844
```

```
# GDP outliers on the log scale (Tukey's method)
log_q <- quantile(analysis_data$log_GDP, c(0.25, 0.75))
log_iqr <- IQR(analysis_data$log_GDP)
sum(analysis_data$log_GDP < log_q[1] - 1.5 * log_iqr |
    analysis_data$log_GDP > log_q[2] + 1.5 * log_iqr)
```

```
## [1] 0
```

```
# The transformed variable is kept alongside the original
dim(analysis_data)
```

```
## [1] 10329     9
```

We stored the log10 version as a new variable, log_GDP, which served the following purposes:

- **Reducing skewness:** skewness fell from 4.48 to 0.10. In the histograms, the values changed from being squeezed against zero with a long tail to the right to a single hill centred around 3 (about $1,000). After being transformed, the new shape is flatter than a perfect bell curve, but much closer to a normal distribution.
- **Turning a non-linear relationship into a linear one:** before the transformation, life expectancy rises steeply at low GDP levels and then levels off, so the points form a curve that a straight line, fitted with lm() and drawn with abline(), fits poorly. After, the points follow a roughly straight line. (Additionally, we found that the correlation between GDP per capita and life expectancy, calculated with cor(), increased from 0.527 to 0.844, so a straight line now explains about 71% of the differences in life expectancy, compared with 28% before).

The transformation also dealt with the GDP outliers identified in Section 9: on the log scale, Tukey's method finds no outliers and life expectancy was left unchanged because its skewness (−0.68) is only mild. Finally, The dataset now contains 10,329 observations and nine variables, including the new log_GDP variable.

11. Reflection on Challenges and Ease

Challenges faced

- **Choosing datasets:** Several options we considered had weak links between the datasets or files we could not access and after close inspection of different files and checking their relationships before committing, they led us to the World Bank datasets, which we believe to meet every requirement.
- **Missing values:** We found that GDP was missing in 18% of rows, well above the 5%, which often means deletion for safety. However, we first investigated where the gaps were (mostly the 1960s and countries that did not yet exist) before deciding that deletion was more honest than inventing values.

- **Outliers:** The methods we intended to use disagreed as Tukey's method flagged 13% of GDP values, while the MVN check flagged far more because of the skewness. Further Testing capping showed that it would raise 839 GDP values, which helped us justify keeping the outliers.

Ease and strengths

- Importing the CSV files and reshaping them with `pivot_longer()` was straightforward, as the year columns were a textbook example of untidy data from Module 4.
- The shared Country Code and Year made the join simple once we confirmed there were no duplicates.
- Comparing histograms made the transformation choice easy, as log10 was clearly the best.

Insights and learning

- Every method has assumptions, so checking results with more than one method and testing alternatives led to better decisions.
- Checking dimensions and counts after each step helped us catch problems early, such as the countries lost during the join.
- Next time, we would check that datasets can be linked and accessed before choosing them, and plan the page limit from the start.

12. Presentation Link(s)

- [Tung - s4275232] (https://rmit-arc.instructuremedia.com/embed/8c8c0a76-f305-4ac0-8507-888eccad83e4 (https://rmit-arc.instructuremedia.com/embed/8c8c0a76-f305-4ac0-8507-888eccad83e4))
- [Hoang - s4280335] (https://rmit-arc.instructuremedia.com/embed/c496a45d-abe5-4c10-8b96-d8d02130a705 (https://rmit-arc.instructuremedia.com/embed/c496a45d-abe5-4c10-8b96-d8d02130a705))

Generative AI Usage

We used one generative AI tools in this assessment. **Val** (RMIT University 2026), RMIT's AI chatbot (Model: Claude Sonnet 5): we asked it "How do I manipulate the format of an rmd file (css) to reduce the number of pages for the pdf" and it suggested a smaller font, narrower margins, less space around headings and smaller figures, set through LaTeX options in the YAML header, as it said CSS only works for HTML output. However, We checked this against how we make our PDF, which was by knitting it to HTML and save it as a PDF from Chrome, so the LaTeX options would not change our report, but CSS does. We therefore applied the same ideas with CSS and chunk options instead, kept all code visible because the brief asks for it, then re-knitted and checked that the PDF stayed within 25 pages and that our results did not change.

References

Doornik JA and Hansen H (2008) 'An omnibus test for univariate and multivariate normality', *Oxford Bulletin of Economics and Statistics*, 70(s1):927–939, doi:10.1111/j.1468-0084.2008.00537.x (doi:10.1111/j.1468-0084.2008.00537.x).

Hyndman RJ and Khandakar Y (2008) 'Automatic time series forecasting: the forecast package for R', *Journal of Statistical Software*, 27(3):1–22, doi:10.18637/jss.v027.i03 (doi:10.18637/jss.v027.i03).

Komsta L (2022) *outliers: Tests for outliers*, R package, version 0.15, doi:10.32614/CRAN.package.outliers (doi:10.32614/CRAN.package.outliers).

Korkmaz S, Goksuluk D and Zararsiz G (2014) 'MVN: An R package for assessing multivariate normality', *The R Journal*, 6(2):151–162, https://journal.r-project.org/articles/RJ-2014-031/RJ-2014-031.pdf (https://journal.r-project.org/articles/RJ-2014-031/RJ-2014-031.pdf).

RMIT University (2026) *Val* [Model: Claude Sonnet 5], accessed 25 October 2026. https://www.rmit.edu.au/students/support-services/study-support/val (https://www.rmit.edu.au/students/support-services/study-support/val)

Wickham H (2025) *stringr: Simple, consistent wrappers for common string operations*, R package, version 1.6.0, doi:10.32614/CRAN.package.stringr (doi:10.32614/CRAN.package.stringr).

Wickham H, François R, Henry L, Müller K and Vaughan D (2026) *dplyr: A grammar of data manipulation*, R package, version 1.2.1, doi:10.32614/CRAN.package.dplyr (doi:10.32614/CRAN.package.dplyr).

Wickham H, Hester J and Bryan J (2026) *readr: Read rectangular text data*, R package, version 2.2.0, doi:10.32614/CRAN.package.readr (doi:10.32614/CRAN.package.readr).

Wickham H, Vaughan D and Girlich M (2025) *tidyr: Tidy messy data*, R package, version 1.3.2, doi:10.32614/CRAN.package.tidyr (doi:10.32614/CRAN.package.tidyr).

World Bank (2026a) *GDP per capita (current US$)* [data set], World Bank Open Data website, accessed 24 October 2026. https://data.worldbank.org/indicator/NY.GDP.PCAP.CD (https://data.worldbank.org/indicator/NY.GDP.PCAP.CD)

World Bank (2026b) *Life expectancy at birth, total (years)* [data set], World Bank Open Data website, accessed 24 October 2026. https://data.worldbank.org/indicator/SP.DYN.LE00.IN (https://data.worldbank.org/indicator/SP.DYN.LE00.IN)